

计算机图形学

课程作业 1：光栅化器

[方林升-102300311]

一、概述

1.1 绘制单色三角形

在 `rasterizer.cpp` 中完成 `RasterizerImp::rasterize_triangle()`：以像素中心一次采样，只在三角形包围盒内测试，边界点视为命中，支持任意顶点顺序，避免逐像素全屏扫描以保证效率。完成后应能正确渲染由单色多边形三角化后的 SVG。

1.2 超采样实现反走样

引入并管理 `sample_buffer`（随窗口与采样率动态分配），将三角形写入子采样缓冲（按 $\sqrt{\text{sample_rate}} \times \sqrt{\text{sample_rate}}$ 网格），最后在 `resolve_to_framebuffer()` 中下采样到 8-bit 帧缓冲。必要时调整 `fill_pixel()` 让点/线写入所有子样本以维持可见性。支持用键 `-/=` 调整采样率。

1.3 几何变换

在 `transforms.cpp` 中实现 3×3 （齐次坐标）平移/缩放/旋转三个矩阵接口（`translate/scale/rotate`），使 `svg/transforms/robot.svg` 正确显示；可在此基础上做 GUI 小扩展作为加分。

1.4 质心插值着色

实现 `RasterizerImp::rasterize_interpolated_color_triangle(...)`：三顶点给定颜色，三角形内部用质心坐标做颜色插值，正确绘制色轮等测试用例。

1.5 基于像素采样的纹理映射

实现 `RasterizerImp::rasterize_textured_triangle(...)`，并在 `Texture` 中完成 `sample_nearest` 与 `sample_bilinear` 两种采样方法；默认从 mipmap 第 0 层采样，GUI 按 P 在最近邻与双线性间切换。

1.6 基于 mipmap 的纹理映射

扩展纹理采样以支持 `L_ZERO / L_NEAREST / L_LINEAR` 三种层级策略：在 `Texture::get_level` 中依据 UV 对屏幕坐标的导数 ($du/dx, dv/dx, du/dy, dv/dy$) 计算合适层级；在 `Texture::sample` 中据此选择单层或对相邻层三线性插值；GUI 按 L 切换层级策略。

二、绘制三角形

对三角形的光栅化当作对二维指示函数的采样问题：对屏幕上每个像素，只在它的像素中心 $(x+0.5, y+0.5)$ 取一个样本，如果样本点在三角形内就着色，否则不着色。

判断“在三角形内”的方法采用半平面/边函数 (edge function)。具体做法是用三条边分别构造有符号面积（或等价的隐式直线方程） $L_i(x,y)$ ，样本点对三条边都为“内侧”时即覆盖；这样不依赖三角形顺/逆时针，只需用整体面积的符号把三条边的符号统一起来即可。对于落在边界上的样本，使用 OpenGL/Direct3D 的 Top-Left 规则（“水平最上边”或“非水平的左边”为内，其余为外），可以避免两个相邻三角形在共有边上“重复/遗漏”像素。

为了不去扫描整张屏幕，先计算三角形顶点的最小/最大 x,y ，得到一个轴对齐包围盒 (AABB)，只在这个矩形的整数像素范围里做点-在-三角形测试，并在进入双重循环前把包围盒和帧缓冲大小做一次裁剪与整型化，能把无用测试的数量从“整屏像素”降到“与三角形面积同阶”。

```

1 //Task 1 Code
2 void RasterizerImp::rasterize_triangle(float x0, float y0,
3                                       float x1, float y1,
4                                       float x2, float y2,
5                                       Color color) {
6
7     auto edge = [](float ax, float ay, float bx, float by, float px, float py) {
8         return (px - ax) * (by - ay) - (py - ay) * (bx - ax);
9     };
10
11     auto is_top_left = [](float dx, float dy) {
12         return (dy > 0.0f) || (dy == 0.0f && dx < 0.0f);
13     };
14
15     float minx_f = std::min({x0, x1, x2});
16     float maxx_f = std::max({x0, x1, x2});
17     float miny_f = std::min({y0, y1, y2});
18     float maxy_f = std::max({y0, y1, y2});
19
20     int xmin = static_cast<int>(std::floor(minx_f));
21     int xmax = static_cast<int>(std::ceil (maxx_f)) - 1;
22     int ymin = static_cast<int>(std::floor(miny_f));
23     int ymax = static_cast<int>(std::ceil (maxy_f)) - 1;
24
25     xmin = std::max(0, xmin);
26     ymin = std::max(0, ymin);
27     if (this->width > 0) xmax = std::min<int>(xmax, static_cast<int>(this->width) - 1);
28     if (this->height > 0) ymax = std::min<int>(ymax, static_cast<int>(this->height) - 1);
29
30     float dx01 = x1 - x0, dy01 = y1 - y0;
31     float dx12 = x2 - x1, dy12 = y2 - y1;
32     float dx20 = x0 - x2, dy20 = y0 - y2;
33
34     bool e01_tl = is_top_left(dx01, dy01);
35     bool e12_tl = is_top_left(dx12, dy12);
36     bool e20_tl = is_top_left(dx20, dy20);
37
38     float area2 = edge(x0, y0, x1, y1, x2, y2);
39     int sgn = (area2 >= 0.0f) ? +1 : -1;
40
41     for (int y = ymin; y <= ymax; ++y) {
42         float py = y + 0.5f;
43
44         for (int x = xmin; x <= xmax; ++x) {
45             float px = x + 0.5f;
46
47             float e0 = sgn * edge(x0, y0, x1, y1, px, py);
48             float e1 = sgn * edge(x1, y1, x2, y2, px, py);
49             float e2 = sgn * edge(x2, y2, x0, y0, px, py);
50
51             bool hit =
52                 (e0 > 0.0f || (e0 == 0.0f && e01_tl)) &&
53                 (e1 > 0.0f || (e1 == 0.0f && e12_tl)) &&
54                 (e2 > 0.0f || (e2 == 0.0f && e20_tl));
55
56             if (hit) {
57                 fill_pixel(x, y, color);
58             }
59         }
60     }
61 }

```

图 2.1: 任务 1 实现代码

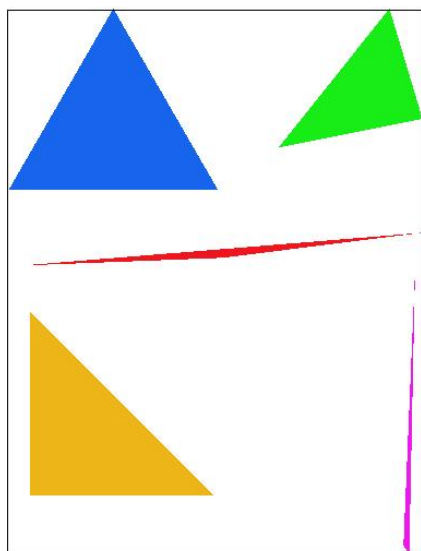


图 2.2: test4.svg 默认视角下的截图

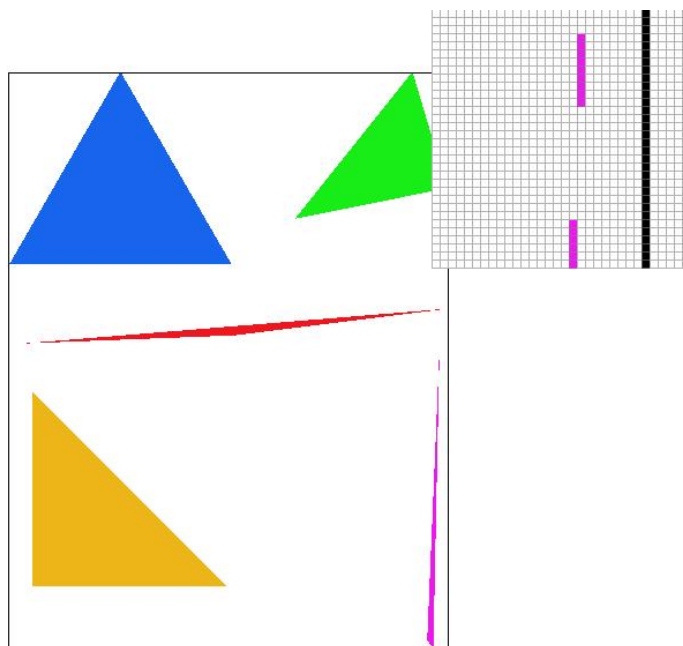


图 2.3: test4.svg 默认视角下像素检查器聚焦在场景某个有趣部分

加分项：

我保留了原算法的包围盒扫描与 **top-left** 规则，但把“对每个像素都重新调用三次 **edge()** 计算叉积/半平面值”的做法改成了增量式评估。核心依据是边函数对像素中心坐标是线性的：对三条边分别有 $e(x,y)=Ax+By+C$ 。因此当光栅化在一行内从 $(x+0.5,y+0.5)$ 移到下一个像素中心 $(x+1.5,y+0.5)$ 时，三条边函数的值只需各自加上一个常量 **A**；换到下一行时，只需各自加上一个常量 **B**。我在进入每一行前，用一次 **edge()** 计算该行起点像素中心的三条初值（**row start**），之后横向只做加法，行尾跳到下一行再各加一次对应的 **B**。

我把所有与边无关、但在原实现里每像素都会重复计算的量全部上提：三条边的方向向量 (dx,dy) 、对应的 **top-left** 判定以及统一朝向所需的符号 **sgn** 都在循环外一次性算出。这样，内层循环只剩下三条边函数的比较（包含等于零时的 **top-left** 处理）与命中后的 **fill_pixel**。这避免了大量的 **lambda** 调用与重复的乘法/减法，显著减少了指令数与寄存器压力，也让分支预测更稳定。

再次，从复杂度与代价上看，原代码每个像素需要对三条边各做一整次叉积（多次乘加/减法）并伴随三次函数调用；现在每个像素横向前进仅做 3 次浮点加法来更新三条边函数，再做几次比较即可。整行换到下一行也只是再做 3 次加法来“重定位”行首状态。对于大三角形，这能把运算密度从“乘法为主”降到“加法为主”，通常带来明显的吞吐提升；同时函数调用次数从“每像素 3 次”降为“每行 0 次”，进一步减少了开销。

最后，正确性方面，采样点仍取像素中心，包围盒裁剪仍保证不做无关区域的检查，**top-left** 规则确保边界像素选择一致，不会出现缝隙或双填充。也就是说，这次优化只改变计算路径与代价，不改变几何判定与光栅结果。如果还要继续挖潜，可以在我现在的增量框架上加两点：行内“先找第一个命中→连续填充直到失败”的 **early-in/out**，以及 $4\times 4/8\times 8$ 的分块（整块全内/全外快速判定，部分覆盖再回落到像素级）。这些都与增量式边函数天然兼容。

```

1 void RasterizerImp::rasterize_triangle(float x0, float y0,
2                                       float x1, float y1,
3                                       float x2, float y2,
4                                       Color color) {
5     auto edge = [](float ax, float ay, float bx, float by, float px, float py) {
6         return (px - ax) * (by - ay) - (py - ay) * (bx - ax);
7     };
8
9     auto is_top_left = [](float dx, float dy) {
10         return (dy > 0.0f) || (dy == 0.0f && dx < 0.0f);
11     };
12
13     float minx_f = std::min({x0, x1, x2});
14     float maxx_f = std::max({x0, x1, x2});
15     float miny_f = std::min({y0, y1, y2});
16     float maxy_f = std::max({y0, y1, y2});
17
18     int xmin = static_cast<int>(std::floor(minx_f));
19     int xmax = static_cast<int>(std::ceil(maxx_f)) - 1;
20     int ymin = static_cast<int>(std::floor(miny_f));
21     int ymax = static_cast<int>(std::ceil(maxy_f)) - 1;
22
23     xmin = std::max(0, xmin);
24     ymin = std::max(0, ymin);
25     if (this->width > 0) xmax = std::min<int>(xmax, static_cast<int>(this->width) - 1);
26     if (this->height > 0) ymax = std::min<int>(ymax, static_cast<int>(this->height) - 1);
27     if (xmax < xmin || ymax < ymin) return;
28
29     float dx01 = x1 - x0, dy01 = y1 - y0;
30     float dx12 = x2 - x1, dy12 = y2 - y1;
31     float dx20 = x0 - x2, dy20 = y0 - y2;
32
33     bool e01_tl = is_top_left(dx01, dy01);
34     bool e12_tl = is_top_left(dx12, dy12);
35     bool e20_tl = is_top_left(dx20, dy20);
36
37     float area2 = edge(x0, y0, x1, y1, x2, y2);
38     int sgn = (area2 >= 0.0f) ? +1 : -1;
39
40     const float A0 = sgn * (dy01);
41     const float B0 = sgn * (x0 - x1);
42     const float A1 = sgn * (dy12);
43     const float B1 = sgn * (x1 - x2);
44     const float A2 = sgn * (dy20);
45     const float B2 = sgn * (x2 - x0);
46
47     const float start_px = xmin + 0.5f;
48     float py = ymin + 0.5f;
49
50     float e0_row = sgn * edge(x0, y0, x1, y1, start_px, py);
51     float e1_row = sgn * edge(x1, y1, x2, y2, start_px, py);
52     float e2_row = sgn * edge(x2, y2, x0, y0, start_px, py);
53
54     for (int y = ymin; y <= ymax; ++y) {
55         float e0 = e0_row, e1 = e1_row, e2 = e2_row;
56
57         for (int x = xmin; x <= xmax; ++x) {
58             const bool hit =
59                 (e0 > 0.0f || (e0 == 0.0f && e01_tl)) &&
60                 (e1 > 0.0f || (e1 == 0.0f && e12_tl)) &&
61                 (e2 > 0.0f || (e2 == 0.0f && e20_tl));
62
63             if (hit) fill_pixel(x, y, color);
64
65             e0 += A0;
66             e1 += A1;
67             e2 += A2;
68         }
69
70         e0_row += B0;
71         e1_row += B1;
72         e2_row += B2;
73         py += 1.0f;
74     }
75 }
76

```

图 2.4: 修改后的三角形光栅化器代码

三、基于超采样的反走样算法

我的实现把每个像素划分为 $N \times N$ 的规则子网格，子样本位于每个小格的中心，对任意一个三角形，我在像素的每个子样本处用半平面（边函数）做命中测试，命中就把该子样本写入对应颜色。等所有几何体都在子样本分辨率下绘制完后，我对同一像素内的 N^2 个子样本做均值并写回可见帧缓冲，相当于先在像素域做 1 像素宽盒式预滤波、再下采样的数值近似，这正是课件里

“Supersampling: 采样多点，再平均”的实现思路。

为了承载这些子样本，我引入了一个一维的颜色采样缓冲 `sample_buffer`，它按像素打平存储所有子样本：总长度为 `width * height * sample_rate`，像素 (x,y) 的第 `sub_id` 个子样本的线性下标是 $((y * \text{width} + x) * \text{sample_rate} + \text{sub_id})$ 。这个缓冲与显示用的 `rgb_framebuffer_target` 分离；前者是高分辨率的子样本颜色池，后者只存最终显示的每像素 RGB。相关接口和注释在头文件中已经约定了采样率、缓冲大小与“将子样本合成为帧缓冲”的职责划分。

为什么超采样有用，可以从直觉与信号处理两端来理解。直觉上，一个像素想表达的是其覆盖区域的平均值；三角形边缘穿越像素时，真实覆盖度在 0 和 1 之间，多点采样后取平均就能把覆盖度转换为中间灰阶，从而明显减轻锯齿。频域上，走样源自高于奈奎斯特频率的成分被稀疏采样折叠到低频；正确做法是采样前用低通滤波抑制高频。1 像素宽的盒式滤波是实用的近似，而“在像素内取多点并平均”就是对这个盒式预滤波的数值实现、。

在光栅化管线上，我做了三处关键改动。第一，初始化、切换采样率或目标帧缓冲尺寸时，都会按 `width * height * sample_rate` 重新分配并清空 `sample_buffer`，确保采样率和画布变化时缓冲一致。第二，三角形阶段不再直接写像素，而是对像素内部的所有子样本逐一测试并写入 `sample_buffer`；点和线为了简洁，则把该像素的所有子样本设为同一颜色以避免边框缝隙，这是起始代码注释中允许的简化。第三，在所有图元栅格化结束后，通过 `resolve_to_framebuffer` 把每个像素的 N^2 子样本做盒式平均并写回可见帧缓冲，从而得到平滑边缘。

```

1 static inline float edge_function(float x0, float y0,
2                                 float x1, float y1,
3                                 float px, float py) {
4     return (px - x0) * (y1 - y0) - (py - y0) * (x1 - x0);
5 }
6
7 static inline bool inside_triangle(float x0, float y0,
8                                   float x1, float y1,
9                                   float x2, float y2,
10                                  float px, float py) {
11     float e0 = edge_function(x0, y0, x1, y1, px, py);
12     float e1 = edge_function(x1, y1, x2, y2, px, py);
13     float e2 = edge_function(x2, y2, x0, y0, px, py);
14     const float eps = 1e-7f;
15     bool all_pos = (e0 >= -eps) && (e1 >= -eps) && (e2 >= -eps);
16     bool all_neg = (e0 <= eps) && (e1 <= eps) && (e2 <= eps);
17     return all_pos || all_neg;
18 }
19
20 static inline size_t sample_index(size_t x, size_t y,
21                                  size_t width,
22                                  unsigned int sample_rate,
23                                  unsigned int sub_id) {
24     return ( (y * width + x) * (size_t)sample_rate ) + sub_id;
25 }

```

图 3.1: 一些要用到的辅助函数

```

1 void RasterizerImp::fill_pixel(size_t x, size_t y, Color c) {
2     if (x >= width || y >= height) return;
3     for (unsigned int s = 0; s < sample_rate; ++s) {
4         sample_buffer[sample_index(x, y, width, sample_rate, s)] = c;
5     }
6 }

```

图 3.2: 对 fill_pixel 函数的修改

```

1 void RasterizerImp::rasterize_triangle(float x0, float y0,
2   float x1, float y1,
3   float x2, float y2,
4   Color color) {
5
6   float minx_f = floor(min(x0, min(x1, x2)));
7   float maxx_f = ceil (max(x0, max(x1, x2)));
8   float miny_f = floor(min(y0, min(y1, y2)));
9   float maxy_f = ceil (max(y0, max(y1, y2)));
10
11  int minx = max(0, (int)minx_f);
12  int maxx = min((int)width - 1, (int)maxx_f - 1);
13  int miny = max(0, (int)miny_f);
14  int maxy = min((int)height - 1, (int)maxy_f - 1);
15
16  int N = (int)roundf(sqrtf((float)sample_rate));
17  N = max(1, N);
18  float invN = 1.0f / (float)N;
19
20  for (int y = miny; y <= maxy; ++y) {
21    for (int x = minx; x <= maxx; ++x) {
22
23      for (int sy = 0; sy < N; ++sy) {
24        for (int sx = 0; sx < N; ++sx) {
25
26          float px = (float)x + (sx + 0.5f) * invN;
27          float py = (float)y + (sy + 0.5f) * invN;
28
29          if (inside_triangle(x0, y0, x1, y1, x2, y2, px, py)) {
30            unsigned int sub_id = (unsigned int)(sy * N + sx);
31            sample_buffer[sample_index((size_t)x, (size_t)y, width, sample_rate, sub_id)] = color;
32          }
33        }
34      }
35    }
36  }
37 }
38 }

```

图 3.3: 对 rasterize_triangle 函数的修改

```

1 void RasterizerImp::set_sample_rate(unsigned int rate) {
2   sample_rate = max(1u, rate);
3   sample_buffer.assign(width * height * (size_t)sample_rate, Color::White);
4 }

```

图 3.4: 对 set_sample_rate 函数的修改

```

1 void RasterizerImp::set_framebuffer_target(unsigned char* rgb_framebuffer,
2   size_t width, size_t height)
3 {
4   this->width = width;
5   this->height = height;
6   this->rgb_framebuffer_target = rgb_framebuffer;
7   sample_buffer.assign(width * height * (size_t)sample_rate, Color::White);
8 }

```

图 3.5: 对 set_framebuffer_target 函数的修改

```

1 void RasterizerImp::resolve_to_framebuffer() {
2     const size_t pix_count = width * height;
3
4     for (size_t pid = 0; pid < pix_count; ++pid) {
5         double r = 0.0, g = 0.0, b = 0.0;
6         size_t base = pid * (size_t)sample_rate;
7         for (unsigned int s = 0; s < sample_rate; ++s) {
8             const Color& c = sample_buffer[base + s];
9             r += c.r; g += c.g; b += c.b;
10        }
11        float inv = 1.0f / (float)sample_rate;
12        Color col((float)(r * inv), (float)(g * inv), (float)(b * inv));
13
14        size_t x3 = 3 * pid;
15        rgb_framebuffer_target[x3 + 0] = (unsigned char)(std::max(0.f, std::min(1.f, col.r)) * 255.f);
16        rgb_framebuffer_target[x3 + 1] = (unsigned char)(std::max(0.f, std::min(1.f, col.g)) * 255.f);
17        rgb_framebuffer_target[x3 + 2] = (unsigned char)(std::max(0.f, std::min(1.f, col.b)) * 255.f);
18    }
19 }
20

```

图 3.6: 对 resolve_to_framebuffer 函数的修改

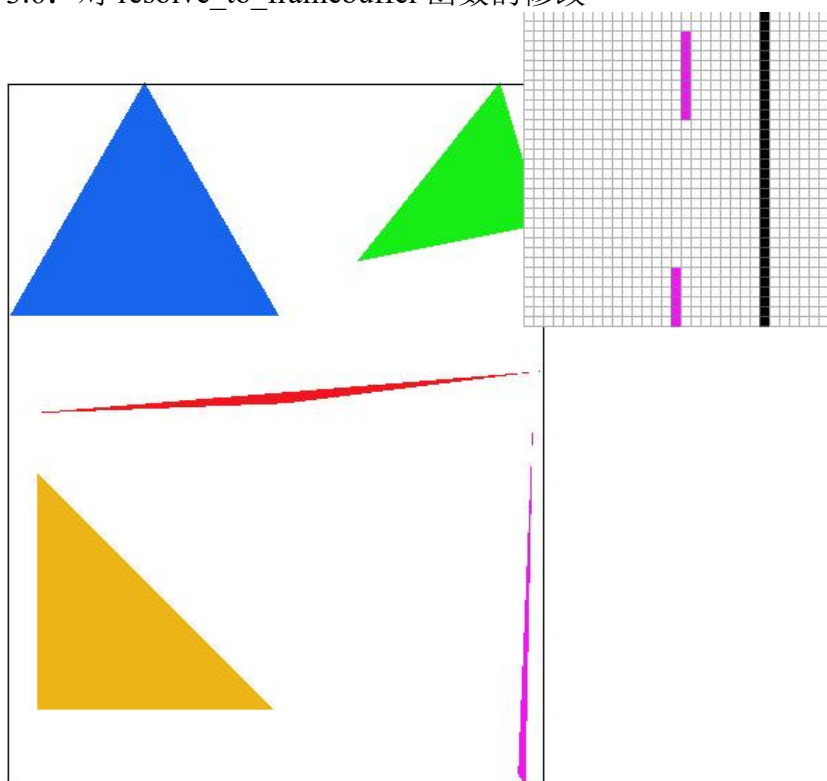


图 3.7: basic/test4.svg 在默认视角下采样率为 1 时的截图

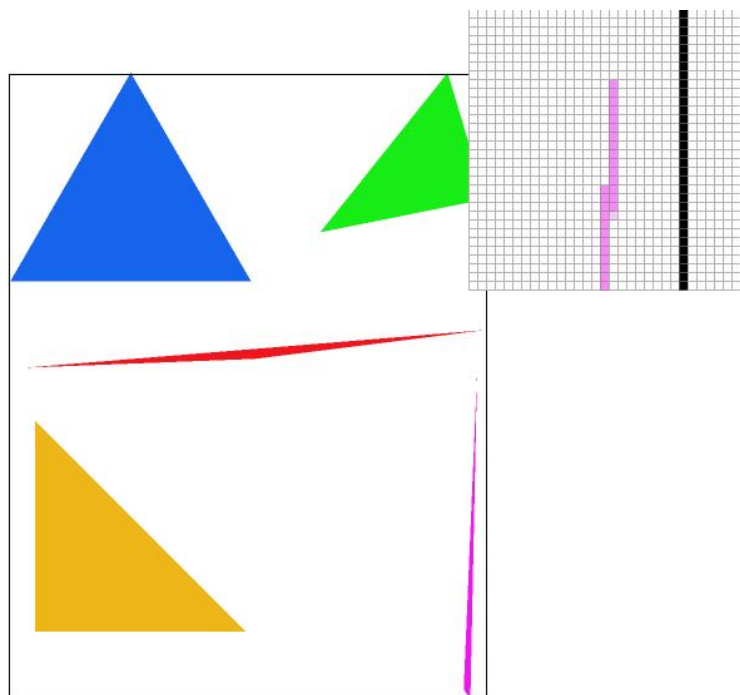


图 3.8: basic/test4.svg 在默认视角下采样率为 4 时的截图

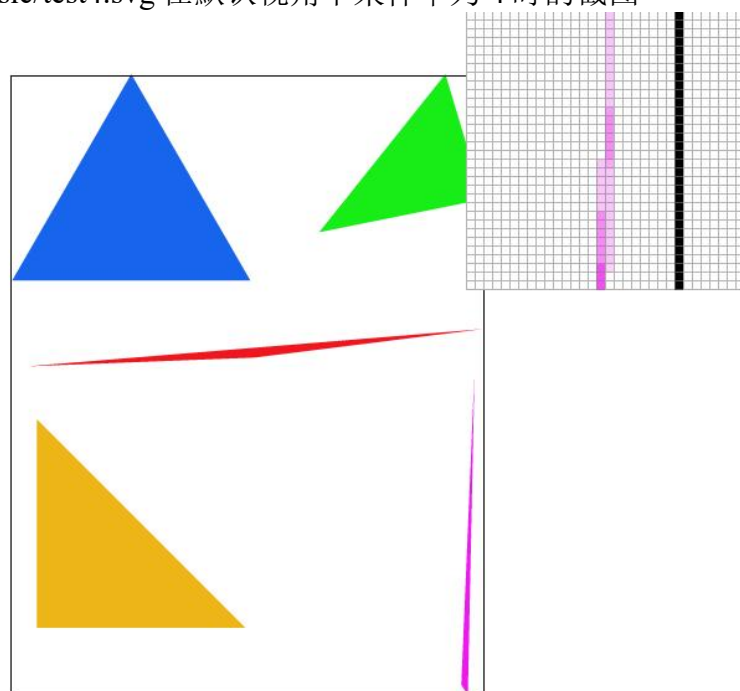


图 3.9: basic/test4.svg 在默认视角下采样率为 16 时的截图

采样率为 1（每像素一个样本）时，像素只能得到“命中/未命中”的二值覆盖，几何边界只能以像素阶梯的方式逼近，所以三角形边缘出现明显锯齿，细线也会出现断裂或粗细忽大忽小的现象。这些都是典型的走样：高频形状被以过低的采样率记录，导致错误的低频模式“折叠”进来。

采样率为 4 时（ 2×2 子样本），每个像素能分辨四个不同的覆盖位置，合成时做均值相当于 1 像素盒式预滤波的更好近似出现，锯齿被显著弱化。

采样率为 16（ 4×4 子样本）时，像素能采到更多位置，覆盖度的量化台阶进一步变细，平均后的像素值更接近真实的面积平均；边缘看起来几乎连续，细线厚度变化平滑而不再闪烁，斜线的亮度/厚度也更线性。这说明更高的子采样密度能更好地抑制走样，但同时代价是填充与合成的计算与内存开销按 N^2 增长。

归根到底，差异来自“像素内平均”的逼近质量：采样率 1 仅用一个点代表整块像素，容易错失边界附近的真实覆盖信息；采样率 4、16 则用更密的子样本去估计该像素对真实图形的平均贡献，均值下采样后就把“部分覆盖”的信息转化为中间灰阶，因此边缘更平滑、细节更稳定。随着采样率提高，估计方差降低、别名成分减少，但收益呈递减，对实时系统通常在质量与成本之间折中选择一个中等采样率。

加分项：

在任务 2 的加分项任务中我实现 MSAA（Multi-Sample Anti-Aliasing）整体思想是：在每个像素内部放置若干固定的子采样点，对几何（这里是三角形）做覆盖判定，命中的子样本就写入同一颜色；最后对该像素内所有子样本的颜色做平均，得到具有抗锯齿效果的最终像素值。由于任务 2 的实心三角形是纯色，子样本不需要重复做着色计算，真正改变的是覆盖率而非着色本身，因此效率高、边缘过渡也更平滑。

首先，我只考虑可选的 1、4、9、16 四种采样率，与课程中 `sample_rate = r*r` 的设定一致。针对每个允许的采样率，我提供了一个像素内采样图案：

1：中心点 (0.5, 0.5)；

4：采用常见的 非规则 4x MSAA pattern（打散规则网格引起的走样）；

9：使用 3×3 的分层中心点（与 SSAA 网格一致，但流程依旧是 MSAA 覆盖率）；

16：采用常见的非规则 16x pattern。在 `set_sample_rate()` 里，任何外部传入的值都会被收敛到上述允许集合，以保证样本访问不越界、`pattern` 与缓冲区大小一致。

几何判定部分，我先用三角形顶点计算像素级包围盒（四舍五入到整像素边界），只在该范围内遍历像素，以减少无效计算。对每个像素，取出与当前采样率匹配的子样本偏移 `offsets[s]`，把它们转换到实际坐标 $(x + dx, y + dy)$ 。然后用边函数（cross product）计算该子样本相对于三条边的有向面积，并做同号性测试（全部非负或全部非正）来判断“是否在三角形内部”。为避免数值误差，我加入了一个很小的 `eps` 容忍度。

缓冲区组织方面，仍然使用框架提供的 `sample_buffer`，其长度为 `width * height * sample_rate`。我实现了一个 `sample_index(x, y, width, sample_rate, s)` 的下标函数，把二维像素坐标与子样本序号映射到一维数组位置。若子样本在三角形内，就把该位置写成三角形颜色；点和线的情形使用 `fill_pixel()` 把该像素下的所有子样本直接写同色，逻辑一致、实现简单。`clear_buffers()` 会把 `rgb_framebuffer_target` 置白、`sample_buffer` 清成白色，以便每帧重新写入。

最终合成在 `resolve_to_framebuffer()` 中完成：我对每个像素的 `sample_rate` 个子样本颜色做算术平均，写回显示帧缓冲的 8-bit RGB。这样得到的像素值本质上就是该像素被三角形覆盖的面积比例（coverage）的线性映射，因此边缘会呈现光滑的灰度过渡，达到抗锯齿的目的。对于 4/16 的非规则样本图案，边缘的

抖动呈现更“随机”的噪声分布，相比规则网格更不容易出现条纹与莫尔效应；而 9（3×3）则兼顾与课程文档、截图中的网格直觉一致性，便于对比与验证。

整体上，这个实现将 MSAA 覆盖率以最低的侵入性融入现有框架：接口不变，纹理采样/重心插值等后续任务的函数也能共用同一缓冲与解析流程。对于任务 2 的纯色三角形，它与基于网格的 SSAA 在数学上都在估计覆盖率，但 MSAA 以更少的着色开销与更优的采样分布提供了质量稳定、性能友好的边缘抗锯齿效果。

```
1 static inline unsigned int clamp_allowed_rate(unsigned int r) {
2     const unsigned int allowed[4] = {1, 4, 9, 16};
3     unsigned int best = allowed[0];
4     unsigned int best_d = std::abs((int)r - (int)allowed[0]);
5     for (unsigned int i = 1; i < 4; ++i) {
6         unsigned int d = std::abs((int)r - (int)allowed[i]);
7         if (d < best_d) { best_d = d; best = allowed[i]; }
8     }
9     return best;
10 }
11
12 static inline vector<pair<float,float>> msaa_pattern(unsigned int n) {
13     switch (n) {
14         case 1:
15             return { {0.5f, 0.5f} };
16         case 4:
17             return {
18                 {0.375f, 0.125f}, {0.875f, 0.375f},
19                 {0.125f, 0.625f}, {0.625f, 0.875f}
20             };
21         case 9: {
22             vector<pair<float,float>> pts;
23             const float s = 1.0f / 3.0f;
24             for (int j = 0; j < 3; ++j)
25                 for (int i = 0; i < 3; ++i)
26                     pts.emplace_back((i + 0.5f) * s, (j + 0.5f) * s);
27             return pts;
28         }
29         case 16:
30             return {
31                 {0.5625f,0.3125f},{0.4375f,0.6875f},{0.8125f,0.5625f},{0.3125f,0.1875f},
32                 {0.1875f,0.8125f},{0.0625f,0.4375f},{0.6875f,0.9375f},{0.9375f,0.0625f},
33                 {0.3750f,0.0625f},{0.6250f,0.1875f},{0.2500f,0.3750f},{0.8750f,0.2500f},
34                 {0.1250f,0.6250f},{0.7500f,0.7500f},{0.5000f,0.8750f},{0.0000f,0.1250f}
35             };
36         default:
37             return { {0.5f, 0.5f} };
38     }
39 }
```

图 3.10: MSAA 要用到的辅助函数

```

1 void RasterizerImp::rasterize_triangle(float x0, float y0,
2                                       float x1, float y1,
3                                       float x2, float y2,
4                                       Color color) {
5     float minx_f = floorf(std::min(x0, std::min(x1, x2)));
6     float maxx_f = ceilf (std::max(x0, std::max(x1, x2)));
7     float miny_f = floorf(std::min(y0, std::min(y1, y2)));
8     float maxy_f = ceilf (std::max(y0, std::max(y1, y2)));
9
10    int minx = std::max(0, (int)minx_f);
11    int maxx = std::min((int)width - 1, (int)maxx_f - 1);
12    int miny = std::max(0, (int)miny_f);
13    int maxy = std::min((int)height - 1, (int)maxy_f - 1);
14    if (maxx < minx || maxy < miny) return;
15
16    const vector<pair<float,float>> offsets = msaa_pattern(sample_rate);
17
18    for (int y = miny; y <= maxy; ++y) {
19        for (int x = minx; x <= maxx; ++x) {
20            for (unsigned int s = 0; s < sample_rate; ++s) {
21                float px = (float)x + offsets[s].first;
22                float py = (float)y + offsets[s].second;
23                if (inside_triangle(x0, y0, x1, y1, x2, y2, px, py)) {
24                    sample_buffer[sample_index((size_t)x, (size_t)y, width, sample_rate, s)] = color;
25                }
26            }
27        }
28    }
29 }

```

图 3.11: 实现 MSAA 对 rasterize_triangle 函数的修改

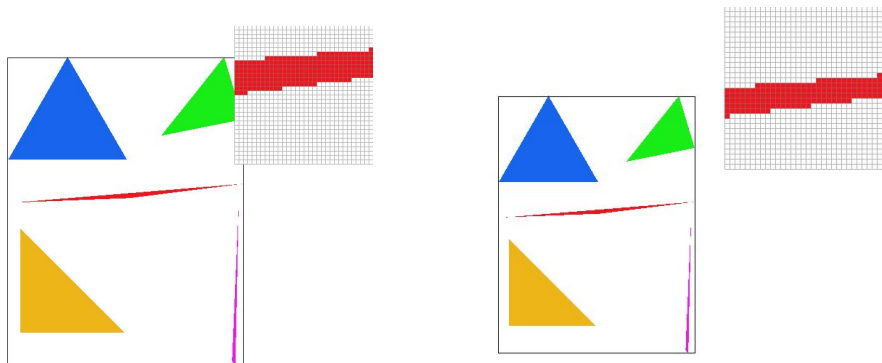


图 3.12: MSAA (左) 与 SSAA (右) 在采样率为 1 时的效果对比

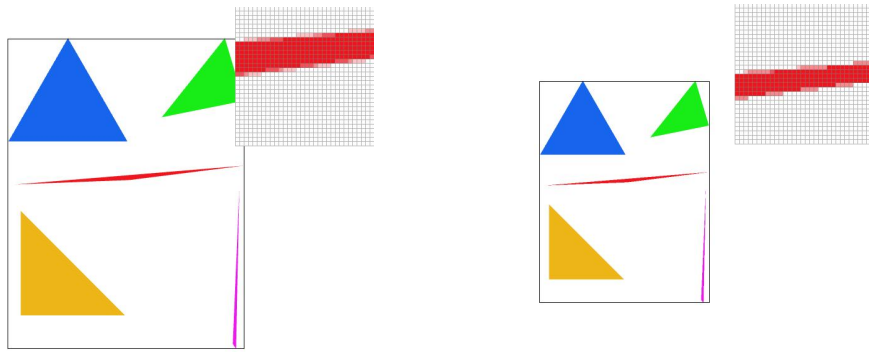


图 3.13: MSAA (左) 与 SSAA (右) 在采样率为 4 时的效果对比

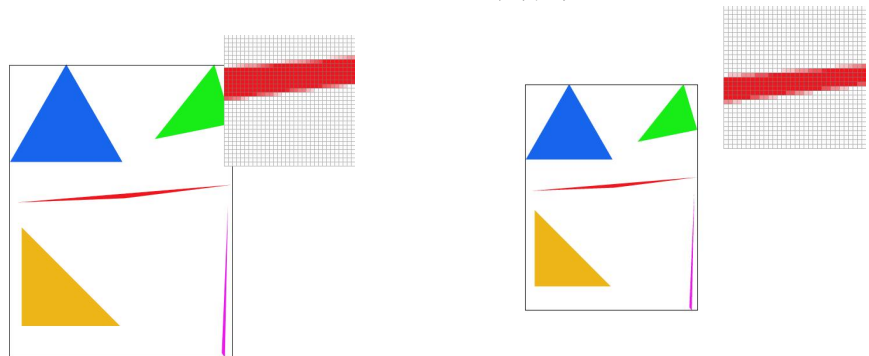


图 3.14: MSAA (左) 与 SSAA (右) 在采样率为 16 时的效果对比

采样率为 1（无抗锯齿）：两者等价：像素只取中心一次。所有斜边都呈现明显的台阶，尤其是图中那条接近水平的红色细三角形，亮一暗边界像“齿条”一样，端点处还会有断续的缺失；整体画面锐利但锯齿最重。

采样率为 4（低倍抗锯齿）：SSAA 的规则 2×2 网格能明显减轻台阶，但在红色细三角形的中段仍能看到周期性的带状条纹：明暗变化会随像素栅格呈周期抖动（网格与几何的“共振”）。MSAA 使用非规则的点位图案，边缘的灰度过渡更“连续”，噪声呈随机细粒，因此在放大视图里能看到更少的条纹与台阶，红条的上下边缘更顺滑。由于本例三角形是纯色，MSAA 采用“覆盖率写色”，无需对每个子样本重复着色，所以在保持相近画质的同时成本更低。

采样率为 16（高倍抗锯齿）：两者都几乎收敛到几何真实覆盖率，整图边缘趋于非常平滑。但差别仍可在红色细三角形的长边缘上放大观察：SSAA 由于采样点严格对齐网格，仍残留极细的、近周期性的微台阶；MSAA 的非规则分布

把这些误差打散成更均匀的微弱随机噪声，主观观感更“糯”，带状感更弱。对极细、接近亚像素厚度的斜边（如右侧那条很窄的洋红色三角）也同样如此。

综合对比可得：

质量：同等样本数下，MSAA 边缘更平滑、条纹更少，尤其在“长且薄、斜率小”的几何上优势更明显；高采样率时，两者都很好，但 MSAA 的残余误差更像随机噪声、主观更自然。

成本：本任务的纯色三角形场景中，MSAA 只做覆盖判定，避免了 SSAA 每个子样本都着色的开销；因此在相近画质下更省算力。

一致性：当采样率升高（如 $16\times$ ）时，两者都会收敛到接近一致的结果；差异主要体现在低到中等采样率与亚像素细几何上，MSAA 更抗“网格共振”。

四、几何变换

我把整个人形看成由多个“局部坐标系里的小部件”组成：一个正方形身躯、两条由上下两段组成的腿、左右各两段的手臂，以及一个正方形的“头”。对每个部件，我都遵循同一个变换顺序：先缩放（定大小）→ 再旋转（定朝向）→ 最后平移（放到画布上的目标位置）。这样做的好处是，缩放与旋转都在各自局部原点附近发生，不会把物体“拖”到意料之外的位置。

身躯部分用单位正方形先做非均匀缩放，得到一个略扁的红色矩形/正方体块；因为它是全身的参照，我没有对它做旋转，只做了一次平移，把它放到画布的大致中心位置，作为肩部与髋部的基准。

手臂我把它拆成左右各两段的细长矩形。每一段先做缩放拉成长条，然后平移到与肩同高的横向位置；中间留出间隙，形成“上臂—空隙—前臂”的视觉节奏。手臂不需要抬起或下垂，所以没有给手臂段加旋转，整体效果就是水平伸展的姿势。

腿部与手臂类似，也拆成左右各两段的细长矩形：先把单位矩形缩放成竖直的长条，之后分别平移到身躯下方作为大腿，再把另一段继续向下平移，形成小腿。上下两段之间留出空隙，看起来像膝盖处的折分，但整体仍保持笔直站立的姿态。

头部起初是一个正方形。我先对它做了一次旋转 45° ，让它变成菱形的样子（对齐到对角线方向）；然后把它平移到身躯上方合适的高度，居中对齐。由于先旋转再平移，头部的菱形朝向不会被后续位移影响，能稳定地“戴”在身体上。

总的来说，小人做出的姿势是：双臂平举、双腿直立、头部微微斜置成菱形。整个画面由若干局部一到一全局的 2D 齐次变换组合而成，每个部件的变换矩阵可写成 $M=T \cdot R \cdot S$ ，保证了大小、朝向与位置都按预期叠加，最终组合成图中的红色小人。

```

1 namespace CGL {
2
3 Vector2D operator*(const Matrix3x3 &m, const Vector2D &v) {
4     Vector3D mv = m * Vector3D(v.x, v.y, 1);
5     return Vector2D(mv.x / mv.z, mv.y / mv.z);
6 }
7
8 Matrix3x3 translate(float dx, float dy) {
9     Matrix3x3 M = Matrix3x3::identity();
10    M[2][0] = dx;
11    M[2][1] = dy;
12    return M;
13 }
14
15 Matrix3x3 scale(float sx, float sy) {
16     Matrix3x3 M = Matrix3x3::identity();
17     M[0][0] = sx;
18     M[1][1] = sy;
19     return M;
20 }
21
22 // The input argument is in degrees counterclockwise
23 Matrix3x3 rotate(float deg) {
24     const float rad = deg * static_cast<float>(M_PI) / 180.0f;
25     const float c = std::cos(rad);
26     const float s = std::sin(rad);
27
28     Matrix3x3 M = Matrix3x3::identity();
29     M[0][0] = c;
30     M[0][1] = s;
31     M[0][2] = 0.0f;
32     M[1][0] = -s;
33     M[1][1] = c;
34     M[1][2] = 0.0f;
35     return M;
36 }
37
38 }
39

```

图 4.1 任务 3 实现代码

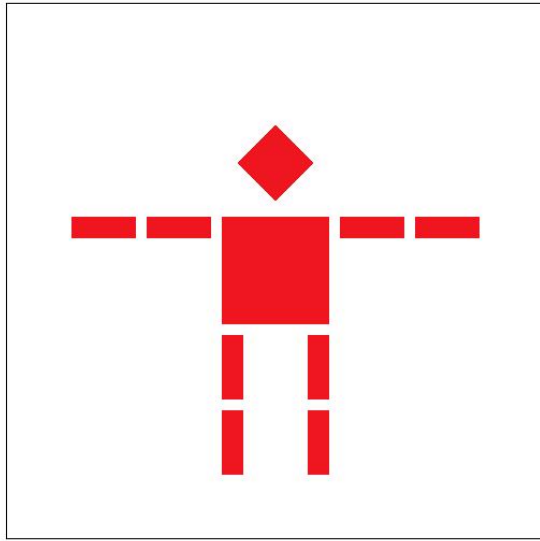


图 4.2 渲染后的 robot.svg

加分项：

修改 robot.svg，使得小人做出举手的样子，渲染后如下图。

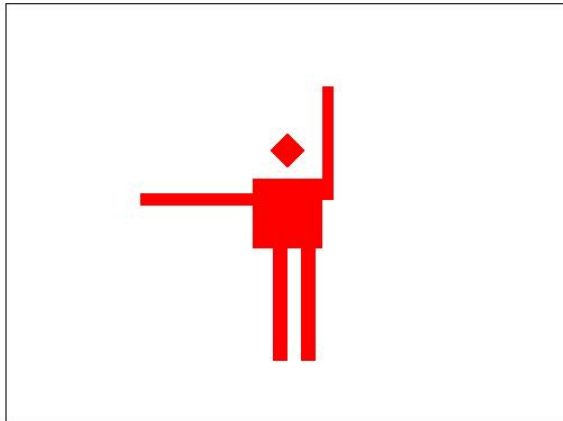


图 4.3 修改后的 robot.svg 小人举手

五、质心坐标

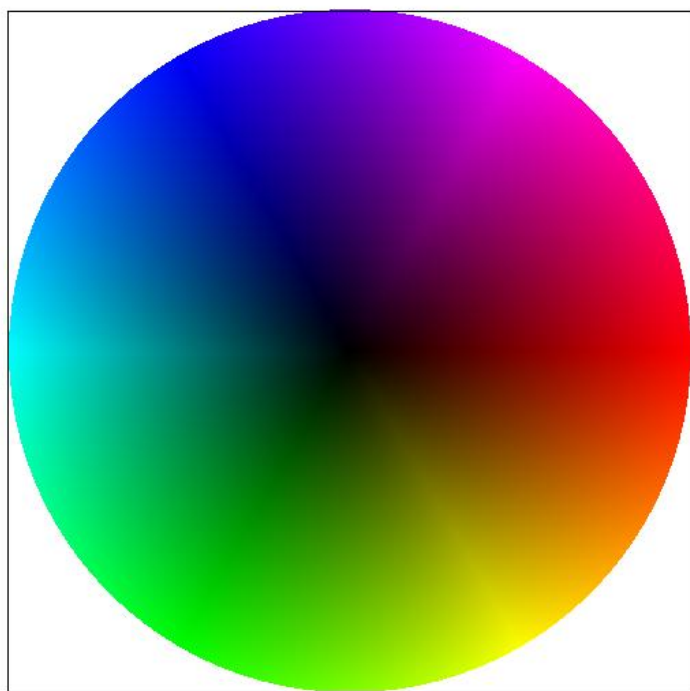


图 5.1 :test7.svg 在默认视角、采样率为 1 时的截图

重心坐标是“用三角形自身来当坐标轴”的办法。对任意一个位于三角形内部（或边上）的点 P ，我们用三个非负权 α, β, γ 重来描述它：

$$\alpha + \beta + \gamma = 1, \quad P = \alpha V_0 + \beta V_1 + \gamma V_2$$

它的几何含义是： α, β, γ 分别对应与对边的带符号面积比（或称边函数值的归一化）。点越靠近某个顶点，该顶点对应的权重就越大；在某顶点处，该顶点权重为 1，另外两个为 0。

因为这些权重在三角形内对线性量保持线性（满足“凸组合”），所以它非常适合做插值：把三角形三个顶点的属性（颜色、法线、纹理坐标等）按 α, β, γ 加权，就能得到内部任意位置的平滑过渡。例如把三个顶点分别设为红、绿、蓝三种颜色：

$$Color(P)=\alpha (1,0,0)+\beta (0,1,0)+\gamma (0,0,1)$$

这样，越靠近红色顶点的区域就越红，靠近绿色顶点就越绿，靠近蓝色顶点就越蓝；三条方向彼此“线性混色”，形成连续的彩色渐变。

实际光栅化时，只需：

- 1) 对三角形求包围盒并遍历像素（或子像素中心）；
- 2) 用边函数（叉积）算出三个未归一化权重 w_0, w_1, w_2 与三角形总面积 A ，令 $\alpha=w_0/A, \beta=w_1/A, \gamma=w_2/A$ ；
- 3) 若 $\alpha, \beta, \gamma \geq 0$ （在同向约定下），像素在三角形内，即可用上式对颜色做加权。这样就得到图中的平滑彩色三角形；把三角形顶点换成不同属性，同样的重心插值也能用于 Gouraud 着色、纹理坐标插值等。

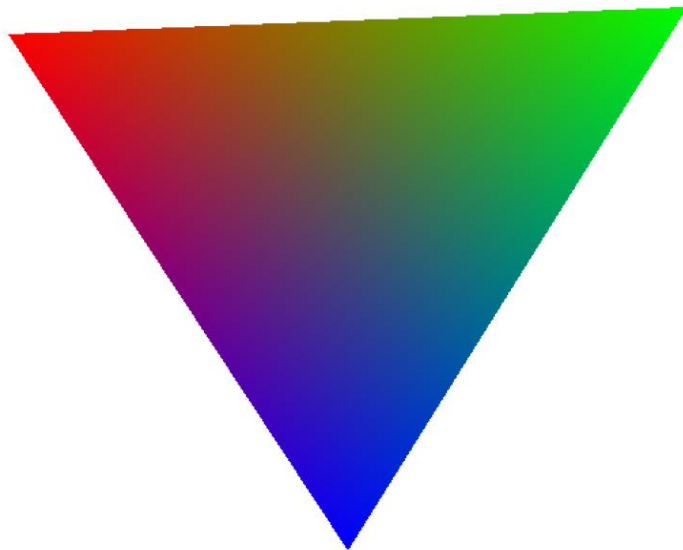


图 5.2：平滑渐变的彩色三角形

```

1 void RasterizerImp::rasterize_interpolated_color_triangle(float x0, float y0, Color c0,
2                                                         float x1, float y1, Color c1,
3                                                         float x2, float y2, Color c2) {
4     auto edge = [](float ax, float ay, float bx, float by, float px, float py) {
5         return (px - ax) * (by - ay) - (py - ay) * (bx - ax);
6     };
7
8     float area = edge(x0, y0, x1, y1, x2, y2);
9     if (area == 0.0f) return;
10
11     auto inside = [&](float w0, float w1, float w2) {
12         if (area > 0) return (w0 >= 0 && w1 > 0 && w2 >= 0) || (w0 > 0 && w1 >= 0 && w2 >= 0);
13         else return (w0 <= 0 && w1 < 0 && w2 <= 0) || (w0 < 0 && w1 <= 0 && w2 <= 0);
14     };
15
16     int min_x = (int)floorf(std::min(std::min(x0, x1), x2));
17     int max_x = (int)ceilf (std::max(std::max(x0, x1), x2));
18     int min_y = (int)floorf(std::min(std::min(y0, y1), y2));
19     int max_y = (int)ceilf (std::max(std::max(y0, y1), y2));
20
21     min_x = std::max(min_x, 0);          max_x = std::min(max_x, (int)width);
22     min_y = std::max(min_y, 0);          max_y = std::min(max_y, (int)height);
23
24     int r = (int)std::round(std::sqrt((double)sample_rate));
25     float step = 1.0f / r;
26
27     auto writeSample = [&](int x, int y, int sx, int sy, const Color& c) {
28         size_t idx = ((size_t)y * width + x) * sample_rate + (size_t)sy * r + sx;
29         sample_buffer[idx] = c;
30     };
31
32     for (int y = min_y; y < max_y; ++y) {
33         for (int x = min_x; x < max_x; ++x) {
34             for (int sy = 0; sy < r; ++sy) {
35                 for (int sx = 0; sx < r; ++sx) {
36                     float px = x + (sx + 0.5f) * step;
37                     float py = y + (sy + 0.5f) * step;
38
39                     float w0 = edge(x1, y1, x2, y2, px, py);
40                     float w1 = edge(x2, y2, x0, y0, px, py);
41                     float w2 = edge(x0, y0, x1, y1, px, py);
42
43                     if (!inside(w0, w1, w2)) continue;
44
45                     float alpha = w0 / area;
46                     float beta  = w1 / area;
47                     float gamma = w2 / area;
48
49                     Color c = c0 * alpha + c1 * beta + c2 * gamma;
50
51                     writeSample(x, y, sx, sy, c);
52                 }
53             }
54         }
55     }
56 }

```

图 5.3: rasterize_interpolated_color_triangle 函数的实现

六、基于像素采样的纹理映射

像素采样（pixel sampling）是在屏幕空间确定一个“将要着色的样本”（通常是像素中心或子像素）后，把它对应到纹理空间，并据此从纹理图像中读取（或重建）一个颜色值的过程。典型做法是：对每个被光栅化覆盖到的屏幕样本 (x,y) ，先用重心插值得到该处的纹理坐标 (u,v) ，再调用纹理的采样函数 `texture.sample(u,v)` 得到颜色并写回帧缓冲。

在这一项任务中，我按“像素采样”的两种常见滤波方式实现了纹理放大（magnification）阶段的重建：最近邻与双线性。最近邻采样把连续坐标 (u,v) 映射到离它最近的整数纹素索引 (i,j) 并直接返回该纹素的颜色；这是最简单的点采样方式，但在缩放时容易产生锯齿和马赛克。

```
1 Color Texture::sample_nearest(Vector2D uv, int level) {
2     auto& mip = mipmap[level];
3
4     double u = std::min(1.0, std::max(0.0, uv.x));
5     double v = std::min(1.0, std::max(0.0, uv.y));
6
7     if (mip.width <= 0 || mip.height <= 0) return Color(1,0,1);
8
9     int tx = (int)std::round(u * (mip.width - 1));
10    int ty = (int)std::round(v * (mip.height - 1));
11
12    tx = std::min<int>(std::max(tx, 0), mip.width - 1);
13    ty = std::min<int>(std::max(ty, 0), mip.height - 1);
14
15    return mip.get_texel(tx, ty);
16 }
```

图 6.1: `sample_nearest` 函数的实现

```

1 Color Texture::sample_bilinear(Vector2D uv, int level) {
2     auto& mip = mipmap[level];
3
4     double u = std::min(1.0, std::max(0.0, uv.x));
5     double v = std::min(1.0, std::max(0.0, uv.y));
6
7     if (mip.width <= 0 || mip.height <= 0) return Color(1,0,1);
8
9     double x = u * (mip.width - 1);
10    double y = v * (mip.height - 1);
11
12    int x0 = (int)std::floor(x);
13    int y0 = (int)std::floor(y);
14    int x1 = std::min<int>(x0 + 1, mip.width - 1);
15    int y1 = std::min<int>(y0 + 1, mip.height - 1);
16
17    double s = x - x0;
18    double t = y - y0;
19
20    Color u00 = mip.get_texel(x0, y0);
21    Color u10 = mip.get_texel(x1, y0);
22    Color u01 = mip.get_texel(x0, y1);
23    Color u11 = mip.get_texel(x1, y1);
24
25    Color u0 = u00 * (1.0 - s) + u10 * s;
26    Color u1 = u01 * (1.0 - s) + u11 * s;
27
28    Color u_ = u0 * (1.0 - t) + u1 * t;
29
30    return u_;
31 }

```

图 6.2: sample_bilinear 函数的实现

```

1 void RasterizerImp::rasterize_textured_triangle(float x0, float y0, float u0, float v0,
2 float x1, float y1, float u1, float v1,
3 float x2, float y2, float u2, float v2,
4 Texture& tex)
5 {
6     int minx = (int)floor(min({ x0, x1, x2 }));
7     int maxx = (int)ceil (max({ x0, x1, x2 }));
8     int miny = (int)floor(min({ y0, y1, y2 }));
9     int maxy = (int)ceil (max({ y0, y1, y2 }));
10
11     minx = max(minx, 0);    maxx = min(maxx, (int)width - 1);
12     miny = max(miny, 0);    maxy = min(maxy, (int)height - 1);
13
14     auto edge = [](float ax, float ay, float bx, float by, float px, float py) {
15         return (px - ax) * (by - ay) - (py - ay) * (bx - ax);
16     };
17     float denom = edge(x0, y0, x1, y1, x2, y2);
18     if (denom == 0) return;
19
20     for (int y = miny; y <= maxy; ++y) {
21         for (int x = minx; x <= maxx; ++x) {
22
23             float px = x + 0.5f, py = y + 0.5f;
24
25             float e0 = edge(x1, y1, x2, y2, px, py);
26             float e1 = edge(x2, y2, x0, y0, px, py);
27             float e2 = edge(x0, y0, x1, y1, px, py);
28
29             bool inside = (e0 >= 0 && e1 >= 0 && e2 >= 0) || (e0 <= 0 && e1 <= 0 && e2 <= 0);
30             if (!inside) continue;
31
32             float alpha = edge(x1, y1, x2, y2, px, py) / denom;
33             float beta  = edge(x2, y2, x0, y0, px, py) / denom;
34             float gamma = 1.0f - alpha - beta;
35
36             Vector2D uv(alpha * u0 + beta * u1 + gamma * u2,
37                        alpha * v0 + beta * v1 + gamma * v2);
38
39             Color c;
40             if (psm == P_NEAREST) {
41                 c = tex.sample_nearest(uv, 0);
42             } else {
43                 c = tex.sample_bilinear(uv, 0);
44             }
45
46             fill_pixel(x, y, c);
47         }
48     }
49 }

```

图 6.3: rasterize_textured_triangle 函数的实现

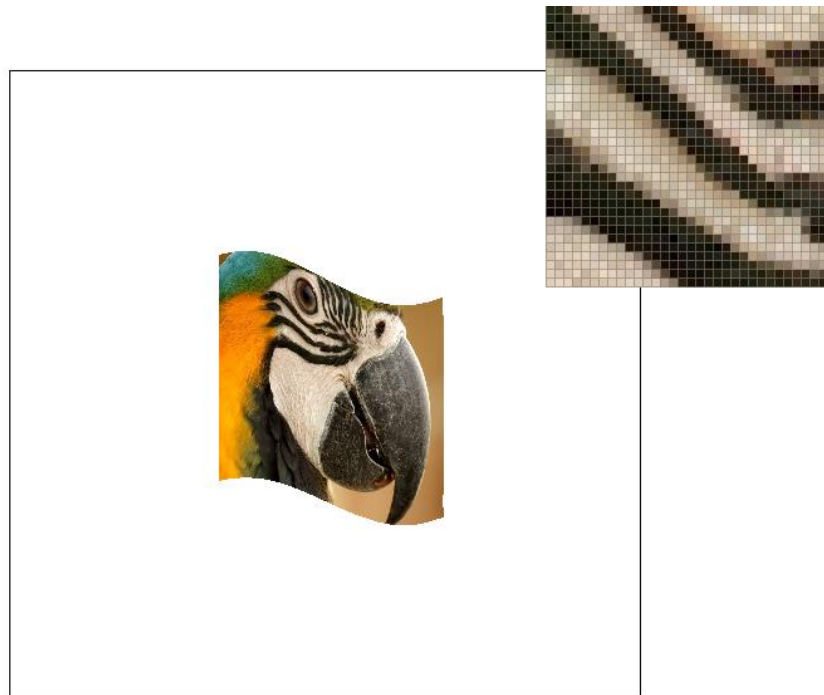


图 6.4: test4.svg, 最近邻采样, 1 像素/采样

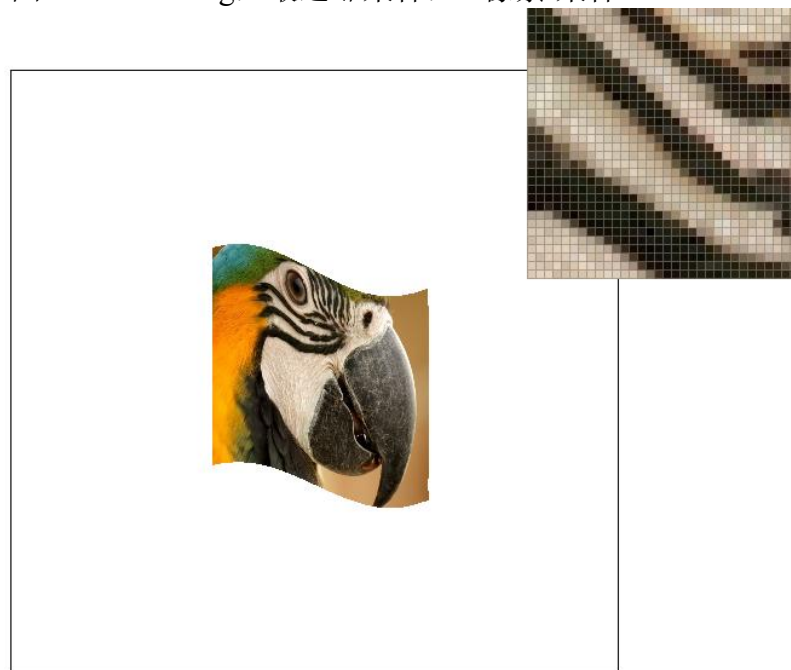


图 6.5: test4.svg, 双线性采样, 1 像素/采样

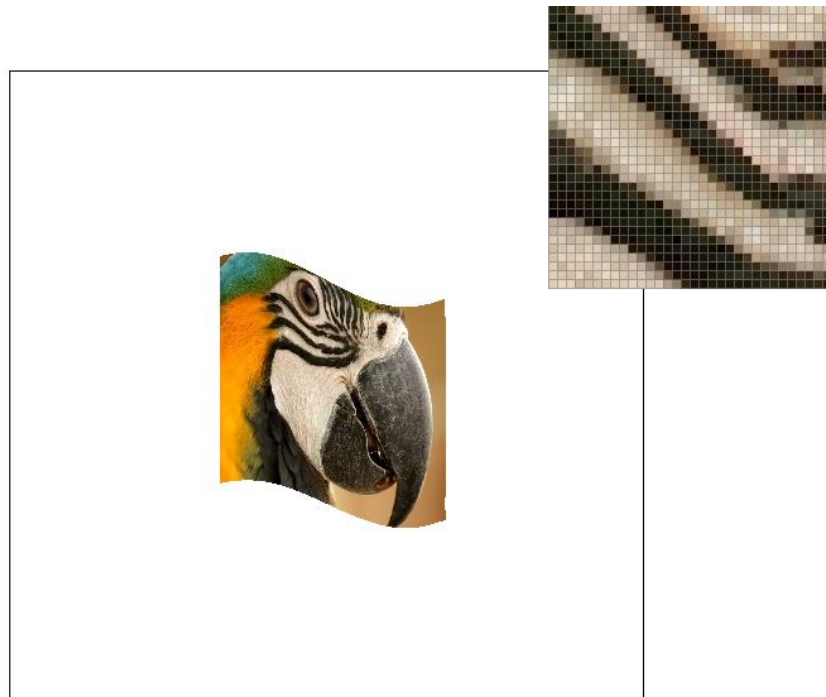


图 6.6: test4.svg, 最近邻采样, 16 像素/采样

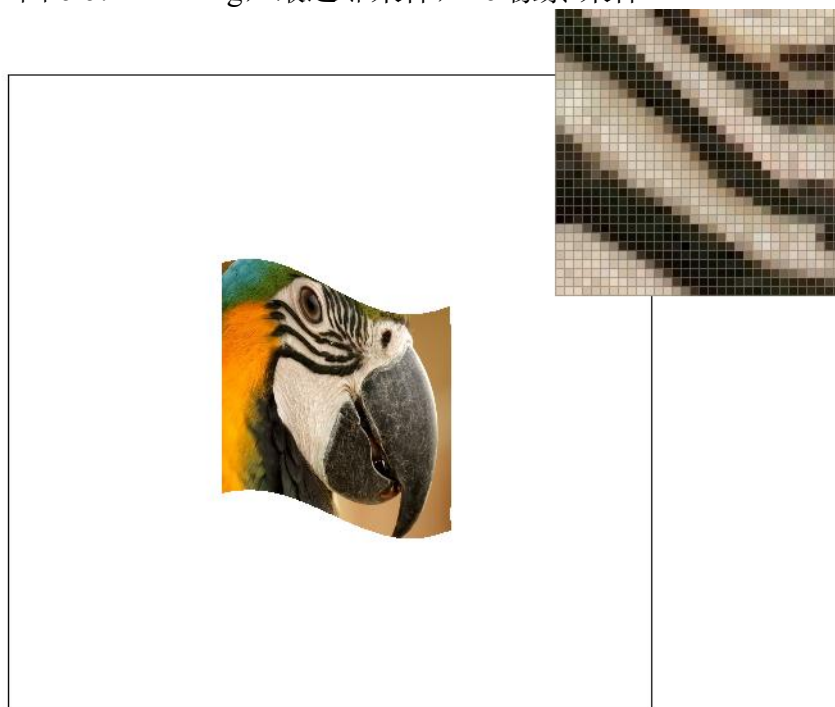


图 6.7: test4.svg, 双线性采样, 16 像素/采样

把这两种采样方式与“每像素 1 次采样（1 spp）/每像素 16 次采样（16 spp）”组合起来看：最近邻+1 spp 最容易出现台阶与马赛克，因为一个屏幕像素只看纹理里的一个点；最近邻+16 spp 会好一些，超采样在屏幕域做了平均，能缓解几何边界附近的锯齿，但同一像素若全部落在同一纹素内，多次采样仍然得到同一个颜色，块感依旧明显。双线性+1 spp 相比最近邻明显更圆滑，细节略软；双线性+16 spp 观感最稳定，既有纹理域的平滑，又有屏幕域的平均，边缘干净、闪烁更少，但最精细的笔画仍会稍微变得柔和。

两种方法的差异在纹理被放大时最大。此时一个屏幕像素对应到纹理中的分辨率很低：最近邻会把单个纹素放大成大块均一色块，边缘呈现明显台阶；双线性用相邻纹素做混合，过渡自然得多，因此主观差距显著。根本原因在于重建核不同：最近邻相当于零阶保持（box 决策），对高频完全不过滤；双线性相当于“帐篷”形低通核，能抑制高频伪影，但以一些锐度为代价。

相反，当纹理被缩小且包含高频细节时，两者都可能出现摩尔纹和闪烁，差异反而没那么大。原因是一个屏幕像素对应到纹理中的面积太大，仅靠最近邻（看 1 个纹素）或双线性（看 2×2 个纹素）都不足以做正确的面积平均——它们解决的是“像素内如何重建颜色”，而不是“跨大面积如何低通采样”。这时应当引入纹理层级与更强的过滤（mipmap/三线性/各向异性），或者显著提高超采样率，先低通再采样，才能从根源上消除走样。

七、MipMap 纹理映射

层级采样（level sampling）是在纹理

空间的尺度与屏幕空间的采样率不匹配（尤其是缩小、minification）时，通过在不同分辨率的纹理层（mipmap）里取样来抑制走样的一类方法。直觉上，当一个屏幕像素覆盖了纹理中的“大块区域”时，如果仍然只从原始最高分辨率去抓少量邻域像素，就会漏掉大量高频信息并产生摩尔纹/闪烁；把纹理预先做成逐级二分的金字塔，每次选择更接近像素足迹面积的那一级取样，就等效于“先低通、再采样”，能显著稳定结果。

在实现上，我采用了作业中给定的三种层级选择策略：

- L_ZERO: 总是从 0 级（原图）采样，行为与任务 5 相同；
- L_NEAREST: 计算出连续的理想层级 D 后四舍五入到最近的整数层（最近层）；
- L_LINEAR: 对 $\lfloor D \rfloor$ 和 $\lceil D \rceil$ 两个相邻层分别采样，然后按 D 的小数部分线性插值（即“三线性”过滤：在“层级轴”上再做一次线性插值）。这些开关通过 GUI 对应到 RasterizerImp 的 lsm 字段，随后由纹理的 sample(...) 根据 lsm 决定取单层还是两层并组合；在被选中的层内，再用 psm 决定“像素重建”方式（最近邻或双线性）。

关键在于怎样得到连续的层级 D。我按课件/作业提示，在光栅化阶段为每个被覆盖的屏幕样本 (x,y) 计算三对纹理坐标：像素中心的 $\mathbf{p}_{uv}$ ，右邻像素 (x+1,y) 的 $\mathbf{p}_{dx_{uv}}$ 与上邻像素 (x,y+1) 的 $\mathbf{p}_{dy_{uv}}$ 。它们用与像素中心相同的重心插值公式从三角形顶点 (u_i, v_i) 求得，然后打包进 SampleParams 传给纹理对象。纹理侧用这三对坐标估计导数 $\partial(u,v)/\partial x$ 、 $\partial(u,v)/\partial y$ ，再乘以基准层（0 级）的宽/高把它们换算成“每屏幕像素跨越的 texel 数”，取两方向范数的最大值 L，最后 $D = \log_2 L$ 。这样得到的 D 是实数：L_NEAREST 取整到最近层，L_LINEAR 则在相邻两层之间插值。

整体流程串起来就是：在 RasterizerImp::rasterize_textured_triangle 中，用边函数/重心坐标测试覆盖并插值出 $\mathbf{p}_{uv}$ ， $\mathbf{p}_{dx_{uv}}$ ， $\mathbf{p}_{dy_{uv}}$ ；把 psm/lsm 和这三对坐标传入 Texture::sample；Texture::get_level 计算 D；Texture::sample 按 lsm 选择单层或两层（并做三线性时的层间插值），在每层内部再按 psm（最近邻或双线性）完成像素重建。这样既保留了任务 5 的像素重建逻辑，又把层级采样纳入统一的数据流，实现了最近层/三线性的 mipmap 纹理映射。

```

1 void RasterizerImp::rasterize_textured_triangle(float x0, float y0, float u0, float v0,
2 float x1, float y1, float u1, float v1,
3 float x2, float y2, float u2, float v2,
4 Texture& tex)
5 {
6     int minx = (int)floor(min({ x0, x1, x2 })), maxx = (int)ceil(max({ x0, x1, x2 }));
7     int miny = (int)floor(min({ y0, y1, y2 })), maxy = (int)ceil(max({ y0, y1, y2 }));
8     minx = max(minx, 0); maxx = min(maxx, (int)width - 1);
9     miny = max(miny, 0); maxy = min(maxy, (int)height - 1);
10
11     auto edge = [](float ax, float ay, float bx, float by, float px, float py) {
12         return (px - ax) * (by - ay) - (py - ay) * (bx - ax);
13     };
14     float denom = edge(x0, y0, x1, y1, x2, y2);
15     if (denom == 0) return;
16
17     for (int y = miny; y <= maxy; ++y) {
18         for (int x = minx; x <= maxx; ++x) {
19             float px = x + 0.5f, py = y + 0.5f;
20
21             float e0 = edge(x1, y1, x2, y2, px, py);
22             float e1 = edge(x2, y2, x0, y0, px, py);
23             float e2 = edge(x0, y0, x1, y1, px, py);
24             bool inside = (e0 >= 0 && e1 >= 0 && e2 >= 0) || (e0 <= 0 && e1 <= 0 && e2 <= 0);
25             if (!inside) continue;
26
27             float alpha = edge(x1, y1, x2, y2, px, py) / denom;
28             float beta = edge(x2, y2, x0, y0, px, py) / denom;
29             float gamma = 1.0f - alpha - beta;
30
31             Vector2D uv(alpha * u0 + beta * u1 + gamma * u2,
32                        alpha * v0 + beta * v1 + gamma * v2);
33
34             auto bary = [&](float qx, float qy, float& a, float& b, float& c) {
35                 a = edge(x1, y1, x2, y2, qx, qy) / denom;
36                 b = edge(x2, y2, x0, y0, qx, qy) / denom;
37                 c = 1.0f - a - b;
38             };
39             float a_dx, b_dx, c_dx, a_dy, b_dy, c_dy;
40             bary(px + 1.0f, py, a_dx, b_dx, c_dx);
41             bary(px, py + 1.0f, a_dy, b_dy, c_dy);
42
43             Vector2D uv_dx(a_dx * u0 + b_dx * u1 + c_dx * u2,
44                          a_dx * v0 + b_dx * v1 + c_dx * v2);
45             Vector2D uv_dy(a_dy * u0 + b_dy * u1 + c_dy * u2,
46                          a_dy * v0 + b_dy * v1 + c_dy * v2);
47
48             SampleParams sp;
49             sp.psm = psm;
50             sp.lsm = lsm;
51             sp.p_uv = uv;
52             sp.p_dx_uv = uv_dx;
53             sp.p_dy_uv = uv_dy;
54
55             Color c = tex.sample(sp);
56             fill_pixel(x, y, c);
57         }
58     }
59 }

```

图 7.1: rasterize_textured_triangle 函数的修改

```

1  Color Texture::sample(const SampleParams& sp) {
2      int maxLevel = (int)mipmap.size() - 1;
3      auto do_pixel = [&](Vector2D uv, int level) -> Color {
4          level = std::max(0, std::min(level, maxLevel));
5          return (sp.psm == P_NEAREST) ? sample_nearest(uv, level)
6                                         : sample_bilinear(uv, level);
7      };
8
9      if (sp.lsm == L_ZERO) {
10         return do_pixel(sp.p_uv, 0);
11     }
12
13     float D = get_level(sp);
14     if (sp.lsm == L_NEAREST) {
15         int L = (int)std::round(D);
16         return do_pixel(sp.p_uv, L);
17     }
18
19     float d = std::min(std::max(D, 0.0f), (float)maxLevel);
20     int L0 = (int)std::floor(d);
21     int L1 = std::min(L0 + 1, maxLevel);
22     float t = d - L0;
23
24     Color c0 = do_pixel(sp.p_uv, L0);
25     Color c1 = do_pixel(sp.p_uv, L1);
26     return c0 * (1.0f - t) + c1 * t;
27 }

```

图 7.2: sample 函数的实现

```

1  float Texture::get_level(const SampleParams& sp) {
2      const MipLevel& base = mipmap[0];
3
4      float dudx = (float)(sp.p_dx_uv.x - sp.p_uv.x);
5      float dvdx = (float)(sp.p_dx_uv.y - sp.p_uv.y);
6      float dudy = (float)(sp.p_dy_uv.x - sp.p_uv.x);
7      float dvdy = (float)(sp.p_dy_uv.y - sp.p_uv.y);
8
9      float ux = dudx * base.width, vx = dvdx * base.height;
10     float uy = dudy * base.width, vy = dvdy * base.height;
11
12     float Lx = std::sqrt(ux * ux + vx * vx);
13     float Ly = std::sqrt(uy * uy + vy * vy);
14     float L = std::max(Lx, Ly);
15
16     return std::max(0.0f, std::log2(std::max(L, 1.0f)));
17 }

```

图 7.3: get_level 函数的实现

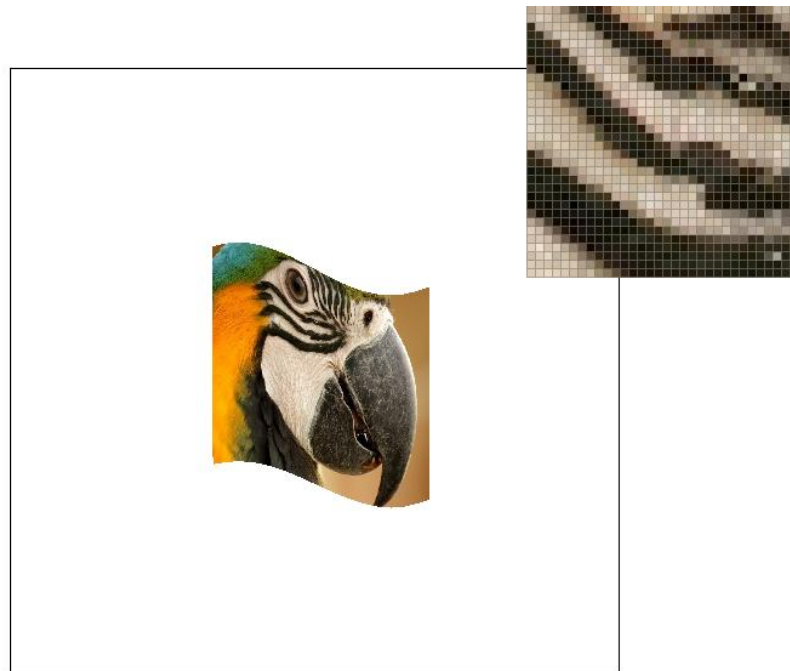


图 7.4: test4.svg, L_ZERO + P_NEAREST, 4 像素/采样

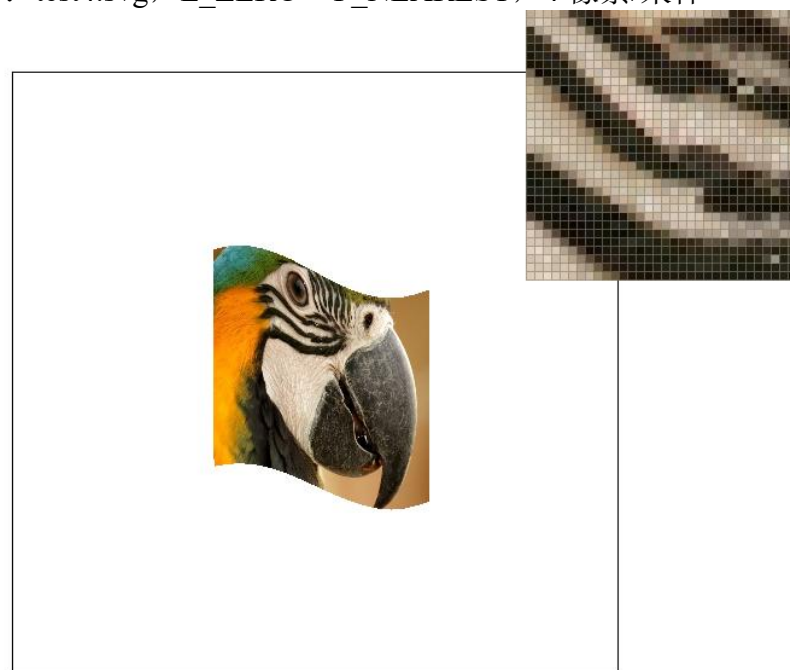


图 7.5: test4.svg, L_ZERO + P_LINEAR, 4 像素/采样

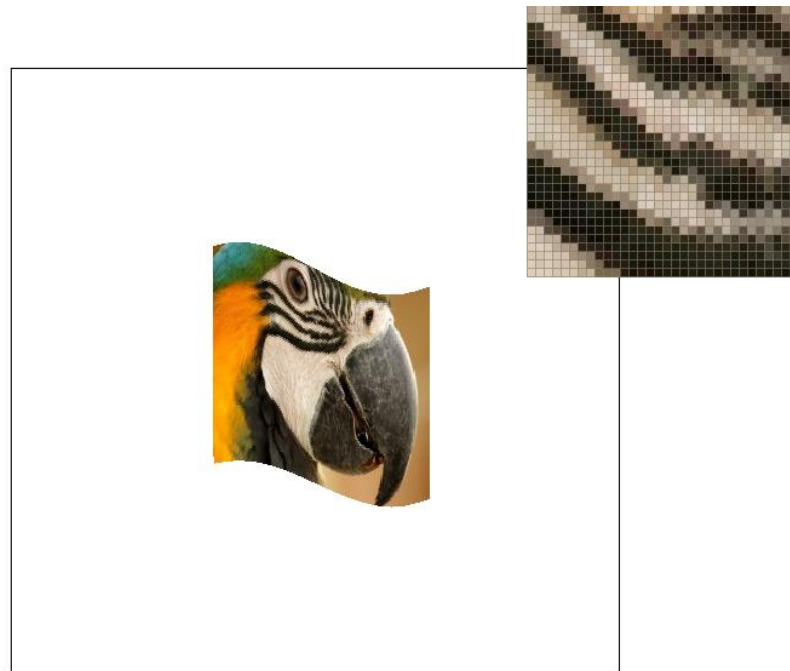


图 7.6: test4.svg, L_NEAREST + P_NEAREST, 4 像素/采样

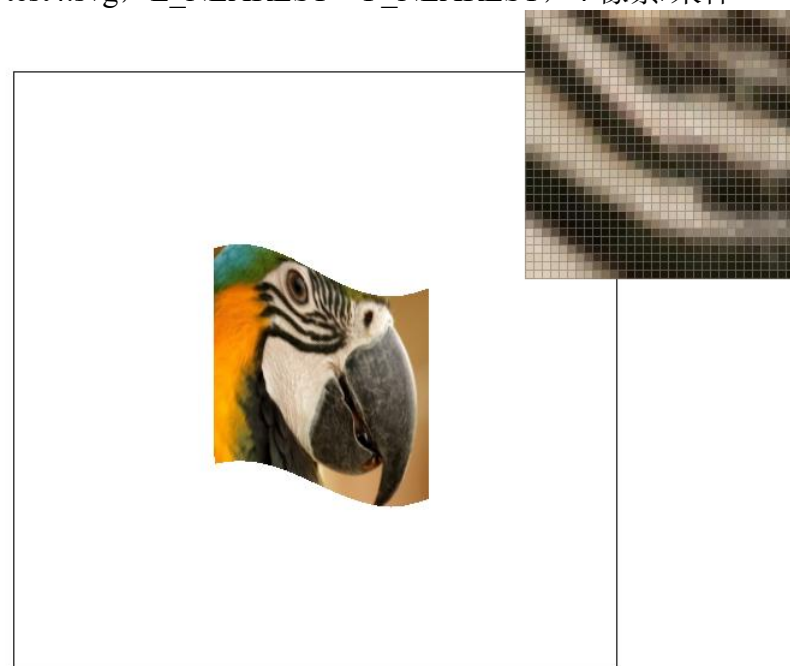


图 7.7: test4.svg, L_NEAREST + P_LINEAR, 4 像素/采样

加分项：

我实现了三线性过滤。做法是先用梯度估计得到一个连续的 mip 层级 D （通常由邻域屏幕样本的纹理坐标差分求出），然后在两张相邻的 mip 层： $\lfloor D \rfloor$ 与 $\lceil D \rceil$ 上分别执行一次像素域重建（最近邻或双线性，取决于 psm），最后按照 D 的小数部分在这两次结果之间线性插值。因为在每一层里已做过双线性（四个纹素、三次 lerp），再加上层与层之间的一次插值，所以整体被称为“三线性”。它的直观含义是：既在纹素网格内做平滑，又在层级方向做平滑，从而避免在层级切换处出现“忽明忽暗”的跳变，同时抑制缩小时的混叠。课程讲义把这一流程明确描述为“在 level D 与 $D+1$ 上各做一次双线性，再按连续 D 做线性插值”。

与之对比，固定 0 层采样完全忽略 D ，一律在 0 层（全分辨率）取样；这等价于只做像素域重建、不做层级选择。放大场景下还凑合，但在缩小（一个屏幕像素对应许多纹素）时会严重混叠，因为它没有利用更低分辨率、已低通的 mip 层。

最近层过滤则是把连续的 D 四舍五入到最近的整数层，只在那一层进行一次像素域重建（最近邻或双线性），不再在层级之间插值。它根据像素的纹理足迹选择更合适的分辨率，混叠明显少于 L_ZERO ；但因为层与层之间是“跳变”切换，随着视距变化会出现轻微的层级闪烁或“阶梯状”变化。

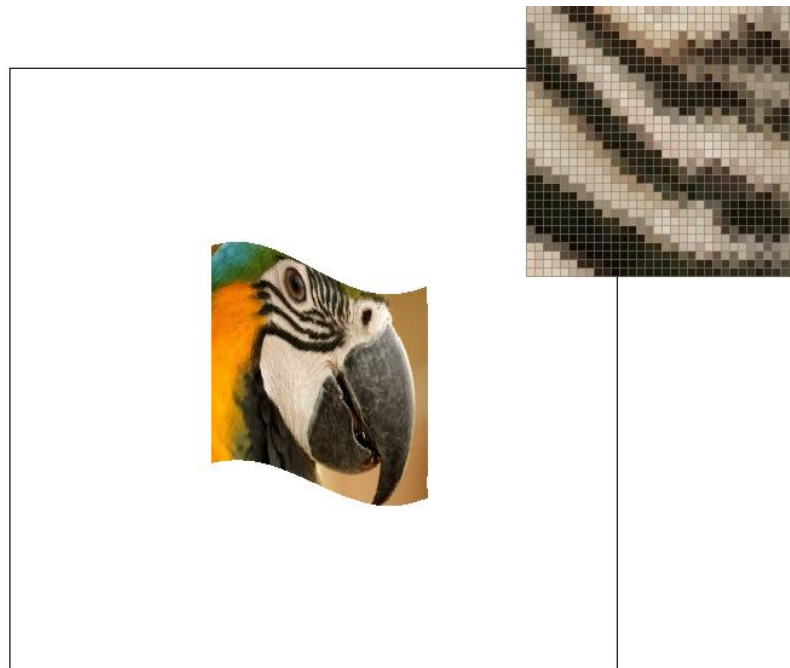


图 7.8: test4.svg, L_LINEAR + P_NEAREST, 4 像素/采样

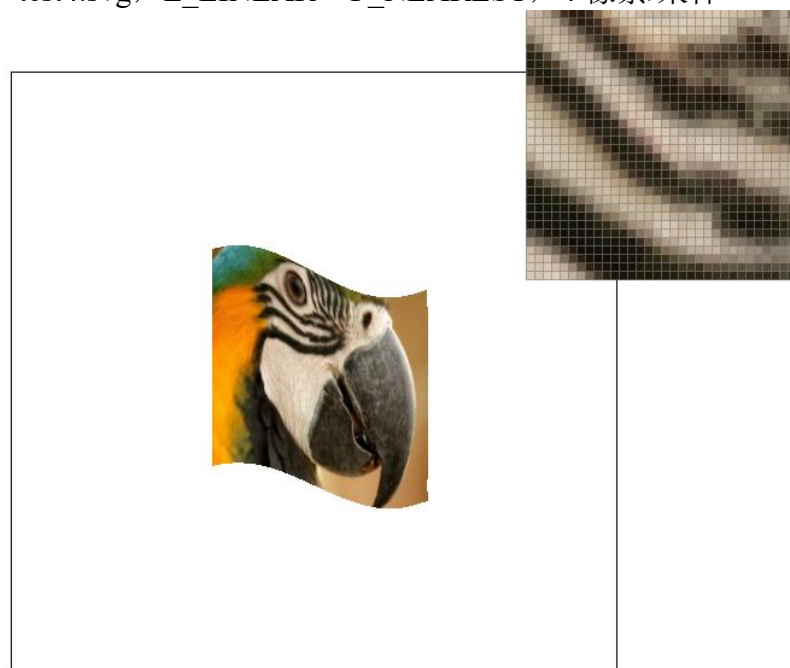


图 7.9: test4.svg, L_LINEAR + P_LINEAR, 4 像素/采样

八、实践与思考

8.1 编程与调试经历

本次编程任务从基础的三角形光栅化开始，逐步扩展到超采样抗锯齿、几何变换、质心插值、纹理映射以及基于 `mipmap` 的多级采样。编程过程中主要使用 C++ 实现，核心逻辑集中在 `rasterizer.cpp`、`transforms.cpp` 和 `texture.cpp` 等文件中。在实现初期，最大的挑战来自边界判定与采样精度问题。例如在三角形光栅化时，若对像素边界处理不当，容易出现相邻三角形间的缝隙或重复填充。通过使用 Top-Left 规则，并结合增量式边函数计算（避免重复叉积计算），最终保证了像素覆盖的一致性与高效性。

在调试超采样与多重采样（MSAA）时，需反复验证采样率与采样位置的匹配关系，尤其是在不同分辨率和采样模式下 `sample_buffer` 的内存分配。通过使用辅助函数验证每个子样本的写入与平均过程，最终实现了稳定的分辨率自适应抗锯齿效果。

此外，几何变换与质心插值部分的调试相对直观，主要在于矩阵乘法顺序与坐标系统一。纹理映射阶段，则需反复测试像素采样与 `mipmap` 层级采样的切换逻辑，确保不同缩放下纹理不会出现明显的闪烁或模糊。

8.2 算法实验效果与优点

实验结果显示，各个算法模块均实现了预期功能。

光栅化阶段：增量式边函数的实现大幅提升了效率，相比逐像素重新计算叉积的方式，运行时间明显缩短，同时保证了与原始算法一致的几何正确性。

超采样与 MSAA 抗锯齿：从 1×1 到 4×4 、 16×16 的采样结果显示，随着采样率提高，边缘锯齿显著减弱，线条平滑过渡自然。MSAA 的非规则采样分布比规则网格抗走样效果更好，边缘噪声呈随机分布，视觉质量更高。

几何变换与质心插值：通过组合平移、旋转、缩放矩阵实现了复杂的局部变换，渲染出的机器人姿态自然。质心插值算法能准确地生成平滑的颜色渐变，验证了重心坐标的线性插值特性。

纹理映射与 mipmap 层级采样：最近邻采样锐利但存在马赛克现象，双线性采样平滑自然；而在缩小纹理时，使用三线性滤波的结果最稳定，有效消除了闪烁与混叠。

8.3 存在的问题与改进思路

尽管各项功能均可运行，但仍存在一些可优化之处：

计算效率方面：超采样在高采样率下计算量迅速上升，可尝试在增量式边函数基础上加入 分块快速裁剪（block-based coverage test）或 early-out 策略，对全内/全外的子块直接跳过遍历，从而进一步降低冗余计算。

内存占用问题：高倍超采样的 sample_buffer 内存需求按采样率平方增长，可采用 动态区域分配 或 tile-based 缓冲 策略，只在局部更新覆盖区域，从而减少总体内存压力。

纹理映射精度：当前的三线性过滤仍以各向同性滤波为主，若在倾斜视角下观察高频纹理，仍可能出现轻微模糊或拉伸，可进一步引入 各向异性过滤（Anisotropic Filtering）来提高纹理细节的方向性保真度。

实时性能：在未来扩展中，可考虑将栅格化与采样部分移植至 GPU 实现，以并行计算加速。

8.4 改进效果总结

优化后的光栅化器在功能与性能上均达到预期。通过增量式边函数与 MSAA 的结合，实现了在有限算力下的高质量抗锯齿；通过 mipmap 的三线性采样，有效解决了纹理缩小时的混叠问题。整体结果图像平滑、层次清晰，渲染过程稳定高效，体现了从基础算法到工程优化的系统化实现思路。